Big Data Project — Complete Guide

Task 3: Web Log URL Categorization | Task 11: Department-Based Partitioning

Platform: Cloudera on VirtualBox | **Team Size:** 6 members

TABLE OF CONTENTS

1. Understanding the Requirements
2. Techniques & Patterns Used
3. Theoretical Steps of the Project
4. Technical Steps (Code Architecture)
5. Full Process Tree
6. GitHub Repository Structure
7. Team Work Assignment (6 Members)
8. Datasets (1 GB+)

---

1. UNDERSTANDING THE REQUIREMENTS

1.1 General Requirements (All Tasks)

Your professor mandates the following for every task submitted:

Requirement	What it Means
Implementation	Use correct pattern classes: CacheMapper, Partitioner, custom WritableComparable
Mapper Logic	Extract the right key-value pairs from each input line
Reducer Logic	Aggregate/enrich values and produce correct output
Driver Class	Wire everything: mappers, reducers, partitioners, DistributedCache, I/O paths
Large File Handling	Must work on ≥ 1 GB datasets
Data Validation	Handle malformed lines, missing fields, non-integer values gracefully
Testing	Verify with sample data AND real large datasets
Submission	`.java` source files, README, sample input/output, documented assumptions
Combiner (Conditional)	Use a Combiner if the aggregation is associative & commutative (reduces shuffle)
Custom Writable (Conditional)	Use WritableComparable when keys/values carry multiple fields

---

1.2 Task 3 — Web Log URL Categorization (Deep Explanation)

What the task is asking

A real web server generates millions of access-log lines per day. Each line looks like:

```
R001,/api/products,120,200
R002,/home,45,200
R003,/api/orders,980,500
R004,/static/logo.png,30,304
```

Each field is: `requestId, urlPath, responseTimeMs, statusCode`

You also have a small lookup file (`url_categories.txt`) that maps URL patterns to human-readable categories:

```
/api/products,API
/api/orders,API
/home,WEB
/static,STATIC
/admin,ADMIN
```

The goal: Classify every log line into a category, then for each category compute:

- How many requests hit it (`requestCount`)
- Average response time in milliseconds (`avgResponseTimeMs`)
- How many requests returned an HTTP error code ≥ 400 (`errorCount`)

Expected Output:

```
API     4   252   1
WEB     10  45    0
STATIC  25  30    0
OTHER   3   200   2
```

Why DistributedCache is the key technique here

The lookup file (`url_categories.txt`) is tiny (a few KB), but there are potentially hundreds of millions of log lines. If the Mapper tries to read the lookup file from HDFS on every single line, it would create enormous I/O overhead.

The solution: Hadoop DistributedCache pushes the small lookup file to every node in the cluster **before** the job starts. Each Mapper then loads it **once** in its `setup()` method into a `HashMap<String, String>` . Every subsequent lookup during `map()` is an O(1) in-memory operation — no disk, no network.

Exact data flow (step by step)

```
url_categories.txt (HDFS) ──→ DistributedCache ──→ Each Mapper Node
                                                        |
                                                   setup(): load into HashMap
                                                        |
Input Log Lines ──→ map() ──→ lookup category from HashMap
                          ──→ emit (category, responseTimeMs + "|" + statusCode)
                                                        |
                                              Shuffle & Sort by category
                                                        |
                              reduce(): for each category:
                                  - count lines            → requestCount
                                  - sum responseTimeMs     → totalTime
                                  - count statusCode ≥ 400 → errorCount
                                  - compute avgResponseTime = totalTime / requestCount
                                         |
```

```
emit: category  requestCount  avgResponseTimeMs  errorCount
```

Handling unknown URLs

If a URL path doesn't match any pattern in the lookup file, the Mapper must emit `"OTHER"` as the category. This is explicitly required.

Error handling requirement

Lines where `responseTimeMs` or `statusCode` cannot be parsed as integers must be **skipped** (not counted, not crashed on). A counter of skipped lines must be logged in the Mapper's `cleanup()` method.

1.3 Task 11 — Department-Based Partitioning (Deep Explanation)

What the task is asking

You have a large employee dataset where each line looks like:

```
E001,IT,5000,Developer,2020-01-15
E002,IT,6000,Senior Developer,2019-03-20
E003,HR,4000,HR Manager,2021-02-10
E004,Sales,4500,Sales Executive,2020-05-15
E005,Finance,5200,Accountant,2020-11-10
```

Fields: `employee_id, department, salary, position, hire_date`

The goal: Route each employee record to a **specific reducer** based on their department, then compute per department:

- Total salary
- Average salary
- Number of employees

Expected Output:

```
IT      Total: 15800   Avg: 5266   Employees: 3
HR      Total: 7800    Avg: 3900   Employees: 2
Sales   Total: 10000   Avg: 5000   Employees: 2
Finance Total: 5200    Avg: 5200   Employees: 1
```

Why Custom Partitioning is the key technique here

In standard MapReduce, the framework uses a **HashPartitioner** by default. It hashes the key and divides by the number of reducers to decide which reducer gets each key. This is fine for random distribution, but it **mixes departments across reducers**, meaning one reducer might get IT and HR records together.

The task requires **one reducer per department**, which guarantees:

- Each output file corresponds to exactly one department
- Processing is isolated per department (no cross-contamination)
- Performance is predictable per group

The solution: Write a custom `Partitioner<Text, Text>` (or use `Configurable`) that builds a `HashMap<String, Integer>` mapping department names to partition numbers, exactly as shown in your `partitioner.txt` example (which maps months to partition numbers using the same pattern).

Connection to your partitioner.txt material

Your existing code in `partitioner.txt` provides `LogProcessorMonthPartitioner`, which:

- Implements `Partitioner<Text, StringPairWritable>`
- Also implements `Configurable` to receive the Hadoop `Configuration`
- In `setConf()`, builds a `HashMap<String, Integer>` of months → partition index
- In `getPartition()`, looks up the month from the value and returns its partition index

Your Task 11 partitioner does exactly the same thing, but for departments instead of months.

StringPairWritable analogy for Task 11

Your `partitioner.txt` also shows `StringPairWritable implements WritableComparable<StringPairWritable>`, which carries two fields (month + year). For Task 11, you can use a simpler approach: the Mapper emits `(department, salary_as_text)` pairs where both are `Text`, so no custom Writable is strictly needed. However, if your professor requires a custom Writable for structured values, you could create an `EmployeeWritable` carrying `salary + position` — a direct adaptation of `StringPairWritable`.

Exact data flow (step by step)

```
Input: employee_id, department, salary, position, hire_date
         |
    map():
      - parse fields
      - validate salary (skip if not integer)
      - emit (department, salary_as_Text)
         |
    CustomDepartmentPartitioner.getPartition():
      - look up department in departmentMap HashMap
      - return partition index
      - ensures all "IT" records → Reducer 0
                 all "HR"    records → Reducer 1
                 all "Sales" records → Reducer 2
                 etc.
         |
    reduce():
      - iterate salary values
      - accumulate: totalSalary, count
      - compute: avgSalary = totalSalary / count
      - emit: department  "Total: X  Avg: Y  Employees: Z"
```

---

2. TECHNIQUES & PATTERNS USED

2.1 Task 3 — Techniques

A. Hadoop DistributedCache

The DistributedCache is a Hadoop mechanism that distributes small, read-only files to every node before a job starts. This is ideal for lookup tables, configuration files, and static reference data.

In the Driver:

```
// Step 1: Upload the file to HDFS (done once, manually or via script)
// hdfs dfs -put url_categories.txt /cache/url_categories.txt

// Step 2: Register the HDFS path as a cache file in the Driver
job.addCacheFile(new URI("/cache/url_categories.txt"));
```

In the Mapper setup():

```java
@Override
protected void setup(Context context) throws IOException, InterruptedException {
    URI[] cacheFiles = context.getCacheFiles();
    // Read the cache file and load into HashMap
    Path cachePath = new Path(cacheFiles[0].getPath());
    FileSystem fs = FileSystem.get(context.getConfiguration());
    BufferedReader reader = new BufferedReader(new InputStreamReader(fs.open(cachePath)));
    String line;
    while ((line = reader.readLine()) != null) {
        String[] parts = line.split(",");
        if (parts.length == 2) {
            urlCategoryMap.put(parts[0].trim(), parts[1].trim());
        }
    }
    reader.close();
}
```

B. CacheMapper Pattern

The CacheMapper performs the lookup during map() after loading the HashMap in setup():

```java
@Override
protected void map(LongWritable key, Text value, Context context) {
    String line = value.toString();
    String[] fields = line.split(",");
    if (fields.length != 4) { skippedCount++; return; }

    String urlPath = fields[1].trim();
    int responseTime, statusCode;
    try {
        responseTime = Integer.parseInt(fields[2].trim());
        statusCode   = Integer.parseInt(fields[3].trim());
    } catch (NumberFormatException e) {
        skippedCount++;
        return;
    }

    // O(1) lookup — no disk I/O
    String category = urlCategoryMap.getOrDefault(urlPath, "OTHER");
    context.write(new Text(category), new Text(responseTime + "|" + statusCode));
}
```

C. CacheReducer with Multi-Field Aggregation

The Reducer accumulates three values for each category key:

```java
@Override
protected void reduce(Text key, Iterable<Text> values, Context context) {
    int requestCount = 0, errorCount = 0;
    long totalResponseTime = 0;
    for (Text val : values) {
        String[] parts = val.toString().split("\\|");
        int responseTime = Integer.parseInt(parts[0]);
        int statusCode   = Integer.parseInt(parts[1]);
        requestCount++;
        totalResponseTime += responseTime;
```

```
        if (statusCode >= 400) errorCount++;
    }
    long avgResponseTime = requestCount > 0 ? totalResponseTime / requestCount : 0;
    context.write(key, new Text(requestCount + "\t" + avgResponseTime + "\t" + errorCount));
}
```

D. Combiner — NOT applicable here

The Combiner cannot be used for Task 3 because:

- `avgResponseTime` is NOT associative: avg(avg(a,b), avg(c,d)) ≠ avg(a,b,c,d)
- You must accumulate raw sums and counts, then divide once in the final Reducer

---

2.2 Task 11 — Techniques

A. Custom Partitioner (direct extension of your partitioner.txt)

Your `LogProcessorMonthPartitioner` from `partitioner.txt` is the blueprint:

```
// FROM YOUR partitioner.txt — adapted for departments
public class DepartmentPartitioner extends Partitioner<Text, Text>
        implements Configurable {

    private Configuration configuration;
    private HashMap<String, Integer> departmentMap = new HashMap<>();

    @Override
    public void setConf(Configuration conf) {
        this.configuration = conf;
        // Read department list from job configuration
        String[] depts = conf.getStrings("departments",
            "IT", "HR", "Sales", "Finance", "Marketing", "Engineering");
        for (int i = 0; i < depts.length; i++) {
            departmentMap.put(depts[i].trim(), i);
        }
    }

    @Override
    public Configuration getConf() { return configuration; }

    @Override
    public int getPartition(Text key, Text value, int numReduceTasks) {
        // key = department name (emitted by Mapper)
        String dept = key.toString().trim();
        Integer partition = departmentMap.get(dept);
        if (partition == null) return 0; // unknown → first reducer
        return partition % numReduceTasks;
    }
}
```

B. Driver Configuration for Partitioner

```
// From your driver.txt adapted with partitioner
Configuration conf = new Configuration();

// Inject the known department list into config so the Partitioner can read it
```

```
conf.setStrings("departments", "IT", "HR", "Sales", "Finance", "Marketing");

Job job = Job.getInstance(conf, "Department Salary Analysis");
job.setPartitionerClass(DepartmentPartitioner.class);
job.setNumReduceTasks(5); // One per department
```

C. Combiner — CAN be used here

For salary totals and counts, the aggregation IS associative and commutative:

- `sum(sum(a,b), sum(c,d)) = sum(a,b,c,d)` ✓
- Running a Combiner locally on each mapper significantly reduces shuffle traffic

You can implement a `SalaryCombiner extends Reducer<Text, Text, Text, Text>` that pre-aggregates salary values before they are sent over the network.

---

3. THEORETICAL STEPS OF THE PROJECT

The project follows a standard Big Data engineering lifecycle:

Phase 1 — Understanding & Planning

1. Read and analyze the task requirements carefully
2. Identify the MapReduce pattern for each task (DistributedCache vs. Custom Partitioner)
3. Decide on data format and schema
4. Choose your dataset

Phase 2 — Environment Setup

1. Start Cloudera VM (VirtualBox)
2. Verify HDFS is running: `hdfs dfs -ls /`
3. Verify YARN is running: `yarn node -list`
4. Create HDFS directories for input, output, and cache files
5. Set up your Java development environment (Eclipse or command line)

Phase 3 — Data Preparation

1. Download or generate your dataset (≥ 1 GB)
2. Inspect the data: check format, encoding, missing values
3. Upload to HDFS: `hdfs dfs -put dataset.txt /input/`
4. For Task 3: create `url_categories.txt` and upload to `/cache/`
5. Verify upload: `hdfs dfs -ls /input/`

Phase 4 — Development

1. Write custom Writable classes (if needed)
2. Write the Mapper class
3. Write the Reducer class
4. Write the Partitioner class (Task 11) or set up DistributedCache (Task 3)
5. Write the Driver class

6. Add error handling and validation

Phase 5 — Compilation & Packaging

1. Compile all `.java` files
2. Package into a `.jar` file
3. Verify the jar contains all class files

Phase 6 — Testing (Small Scale)

1. Create small sample input (10-20 lines)
2. Upload sample to HDFS
3. Run the jar on the small input
4. Verify output matches expected results

Phase 7 — Full Scale Execution (≥ 1 GB)

1. Run the jar on the full dataset
2. Monitor job progress in YARN ResourceManager UI (localhost:8088)
3. Check output files in HDFS

Phase 8 — Validation & Documentation

1. Validate output (spot-check results, verify logic)
2. Write README with steps to reproduce
3. Capture sample input/output files
4. Document assumptions (e.g., department list, URL matching strategy)

---

4. TECHNICAL STEPS (CODE ARCHITECTURE)

4.1 Task 3 — Complete Java Class Structure

```
CacheDriver.java
├── Reads 3 args: inputDir, outputDir, cacheFilePath
├── Calls conf.set("cache.file.path", cacheFilePath)     ← from your slides
├── Validates cache file exists on HDFS (FileSystem.exists())
├── Calls job.addCacheFile(new URI(cacheFilePath))
├── Sets: Mapper=CacheMapper, Reducer=CacheReducer
├── Sets: MapOutputKey=Text, MapOutputValue=Text
├── Sets: OutputKey=Text, OutputValue=Text
├── Sets: NumReduceTasks=1 (as required)
└── Submits and waits

CacheMapper.java
├── setup(context): load url_categories.txt into HashMap<String,String>
├── map(key, value, context):
│   ├── split line by ","
│   ├── validate length == 4, else skippedCount++; return
│   ├── parse responseTimeMs and statusCode as int, else skippedCount++; return
```

```
│   ├── lookup urlPath in HashMap (default: "OTHER")
│   └── emit (category, responseTimeMs + "|" + statusCode)
└── cleanup(context): log skippedCount

CacheReducer.java
└── reduce(category, values, context):
    ├── iterate values: split by "|", accumulate count, sum, errorCount
    ├── compute avgResponseTime = sum / count (integer division)
    └── emit (category, count + "\t" + avg + "\t" + errorCount)
```

HDFS Commands for Task 3:

```bash
# Create directories
hdfs dfs -mkdir -p /task3/input
hdfs dfs -mkdir -p /task3/cache
hdfs dfs -mkdir -p /task3/output

# Upload files
hdfs dfs -put web_logs.txt /task3/input/
hdfs dfs -put url_categories.txt /task3/cache/

# Compile and package
javac -classpath $(hadoop classpath) *.java
jar cf task3.jar *.class

# Run
hadoop jar task3.jar CacheDriver /task3/input /task3/output /task3/cache/url_categories.txt

# Check output
hdfs dfs -cat /task3/output/part-r-00000
```

4.2 Task 11 — Complete Java Class Structure

```
DepartmentDriver.java
├── Reads 2 args: inputDir, outputDir (from your driver.txt pattern)
├── Calls conf.setStrings("departments", "IT","HR","Sales","Finance",...)
├── Sets: Mapper=EmployeeMapper, Reducer=DepartmentReducer
├── Sets: Partitioner=DepartmentPartitioner        ← like LogProcessorDriver in partitioner.txt
├── Sets: MapOutputKey=Text, MapOutputValue=Text
├── Sets: OutputKey=Text, OutputValue=Text
├── Sets: NumReduceTasks = number of departments
└── Submits and waits

EmployeeMapper.java
└── map(key, value, context):
    ├── split by ","
    ├── validate length >= 3, else return
    ├── parse salary as int, else skippedCount++; return
    ├── extract department (fields[1])
    └── emit (department, salary_as_Text)

DepartmentPartitioner.java            ← DIRECT ADAPTATION of partitioner.txt
├── implements Partitioner<Text, Text>
├── implements Configurable
├── setConf(): read "departments" config, build HashMap<String,Integer>
└── getPartition(): look up department name, return partition index
```

```
        getPartition(): look up department name, return partition index

    DepartmentReducer.java
    └── reduce(department, salaries, context):
        ├── iterate: parse salary, accumulate total, count
        ├── handle parse errors (skip bad values)
        ├── compute avg = total / count
        └── emit (department, "Total: X  Avg: Y  Employees: Z")
```

HDFS Commands for Task 11:

```
# Create directories
hdfs dfs -mkdir -p /task11/input
hdfs dfs -mkdir -p /task11/output

# Upload dataset
hdfs dfs -put employees.csv /task11/input/

# Compile and package
javac -classpath $(hadoop classpath) *.java
jar cf task11.jar *.class

# Run
hadoop jar task11.jar DepartmentDriver /task11/input /task11/output

# Check outputs (one file per department/reducer)
hdfs dfs -ls /task11/output/
hdfs dfs -cat /task11/output/part-r-00000   # e.g., IT
hdfs dfs -cat /task11/output/part-r-00001   # e.g., HR
```

---

5. FULL PROCESS TREE

```
Project
│
├── TASK 3: Web Log URL Categorization
│   │
│   ├── [INPUT LAYER]
│   │   ├── web_logs.txt             (1GB+, stored in HDFS /task3/input/)
│   │   │   Format: requestId, urlPath, responseTimeMs, statusCode
│   │   └── url_categories.txt       (small, stored in HDFS /task3/cache/)
│   │       Format: urlPattern, category
│   │
│   ├── [DISTRIBUTED CACHE LAYER]
│   │   └── CacheDriver registers url_categories.txt via job.addCacheFile()
│   │       └── Hadoop pushes file to every TaskTracker node before job starts
│   │
│   ├── [MAP PHASE]
│   │   └── CacheMapper
│   │       ├── setup(): reads cache file → HashMap<urlPattern, category>
│   │       ├── map(): for each log line
│   │       │   ├── validate: 4 fields, numeric responseTime & statusCode
│   │       │   ├── lookup urlPath → category (default: "OTHER")
│   │       │   └── emit: (category, "responseTimeMs|statusCode")
│   │       └── cleanup(): log skippedCount
│   │
│   ├── [SHUFFLE & SORT PHASE]
```

```
│   │   └── Framework groups all values by category key
│   │       e.g., API → [120|200, 980|500, 45|200, ...]
│   │
│   ├── [REDUCE PHASE]
│   │   └── CacheReducer (1 reducer)
│   │       └── reduce(): for each category
│   │           ├── accumulate: requestCount, sumResponseTime, errorCount
│   │           ├── compute: avgResponseTime = sum / count
│   │           └── emit: (category, "count\tavg\terrorCount")
│   │
│   └── [OUTPUT LAYER]
│       └── HDFS /task3/output/part-r-00000
│           Format: category  requestCount  avgResponseTimeMs  errorCount
│
│
└── TASK 11: Department-Based Partitioning
    │
    ├── [INPUT LAYER]
    │   └── employees.csv              (1GB+, stored in HDFS /task11/input/)
    │       Format: employee_id, department, salary, position, hire_date
    │
    ├── [DRIVER CONFIGURATION LAYER]
    │   └── DepartmentDriver
    │       ├── conf.setStrings("departments", "IT","HR","Sales","Finance",...)
    │       ├── job.setPartitionerClass(DepartmentPartitioner.class)
    │       └── job.setNumReduceTasks(N)   // N = number of departments
    │
    ├── [MAP PHASE]
    │   └── EmployeeMapper
    │       └── map(): for each employee record
    │           ├── validate: minimum 3 fields
    │           ├── parse salary as integer (skip if invalid)
    │           └── emit: (department, salary_string)
    │
    ├── [PARTITION PHASE]    ← THE KEY DIFFERENCE FROM DEFAULT HADOOP
    │   └── DepartmentPartitioner
    │       ├── setConf(): build HashMap {IT→0, HR→1, Sales→2, Finance→3, ...}
    │       └── getPartition(): return departmentMap.get(department) % numReducers
    │           ├── All IT records   → Reducer 0
    │           ├── All HR records   → Reducer 1
    │           ├── All Sales records → Reducer 2
    │           └── All Finance records → Reducer 3
    │
    ├── [SHUFFLE & SORT PHASE]
    │   └── Each reducer receives only its department's records
    │
    ├── [REDUCE PHASE]
    │   └── DepartmentReducer (N reducers, one per department)
    │       └── reduce(): for each department
    │           ├── iterate salaries: accumulate total, count
    │           ├── compute: avgSalary = total / count
    │           └── emit: (dept, "Total: X  Avg: Y  Employees: Z")
    │
    └── [OUTPUT LAYER]
        └── HDFS /task11/output/
            ├── part-r-00000   (IT department)
            ├── part-r-00001   (HR department)
```

```
        ├── part-r-00002   (Sales department)
        └── part-r-00003   (Finance department)
```

---

6. GITHUB REPOSITORY STRUCTURE

```
bigdata-project/
│
├── README.md                        ← Project overview, how to run, team members
├── .gitignore                       ← Ignore: *.class, *.jar, target/, output/
│
├── docs/
│   ├── project_requirements.md      ← Paste/summarize the original task specs
│   ├── architecture_diagram.md      ← Describe the MapReduce flow in text/ASCII
│   ├── team_assignments.md          ← Who owns what (mirrors Section 7 below)
│   └── assumptions.md               ← Any design decisions (e.g., URL matching logic)
│
├── task3-web-log-categorization/
│   │
│   ├── README.md                    ← Task 3 specific instructions & run commands
│   │
│   ├── src/
│   │   ├── CacheDriver.java         ← Driver: wires everything, registers cache file
│   │   ├── CacheMapper.java         ← Mapper: loads HashMap, categorizes URLs
│   │   └── CacheReducer.java        ← Reducer: aggregates count, avg, errors
│   │
│   ├── data/
│   │   ├── sample_web_logs.txt      ← Small sample (20-30 lines for testing)
│   │   ├── url_categories.txt       ← The cache/lookup file (urlPattern, category)
│   │   └── expected_output.txt      ← Expected output for the sample input
│   │
│   ├── scripts/
│   │   ├── setup_hdfs.sh            ← Creates HDFS dirs, uploads input & cache file
│   │   ├── compile_and_run.sh       ← Compiles, packages jar, runs hadoop jar
│   │   └── check_output.sh          ← Fetches and displays HDFS output
│   │
│   └── output/
│       └── .gitkeep                 ← Placeholder (actual outputs not committed)
│
├── task11-department-partitioning/
│   │
│   ├── README.md                    ← Task 11 specific instructions & run commands
│   │
│   ├── src/
│   │   ├── DepartmentDriver.java    ← Driver: sets partitioner, num reducers, config
│   │   ├── EmployeeMapper.java      ← Mapper: emits (department, salary)
│   │   ├── DepartmentPartitioner.java ← Custom partitioner (dept → reducer index)
│   │   └── DepartmentReducer.java   ← Reducer: total salary, avg salary, count
│   │
│   ├── data/
│   │   ├── sample_employees.csv     ← Small sample (20-30 lines for testing)
│   │   └── expected_output.txt      ← Expected output for each department
│   │
│   ├── scripts/
│   │   ├── setup_hdfs.sh            ← Creates HDFS dirs, uploads employee dataset
│   │   ├── compile_and_run.sh       ← Compiles, packages jar, runs hadoop jar
```

```
│   │   └── check_output.sh           ← Fetches and displays all reducer output files
│   │
│   └── output/
│       └── .gitkeep
│
└── shared/
    ├── StringPairWritable.java        ← Reusable WritableComparable (from your partitioner.txt)
    └── hadoop-build.xml               ← Ant or Maven build file if used
```

Key `.gitignore` content:

```
*.class
*.jar
/task3-web-log-categorization/output/*
/task11-department-partitioning/output/*
target/
.classpath
.project
.settings/
```

Branching Strategy for 6 Members:

```
main
├── develop                  ← Integration branch (merge here before main)
│   ├── task3/mapper         ← Member 1 works here
│   ├── task3/reducer-driver ← Member 2 works here
│   ├── task3/testing        ← Member 3 works here
│   ├── task11/mapper-reducer ← Member 4 works here
│   ├── task11/partitioner   ← Member 5 works here
│   └── task11/testing-docs  ← Member 6 works here
```

---

7. TEAM WORK ASSIGNMENT (6 MEMBERS)

Division Philosophy

Split by: (a) task ownership, (b) component layer, (c) cross-cutting concerns

Member	Role	Responsibilities	Files to Own
Member 1 — Task 3 Mapper Lead	CacheMapper developer	Write `CacheMapper.java` ; implement `setup()` with DistributedCache loading; implement `map()` with HashMap lookup and error handling; implement `cleanup()` for skipped count; unit-test the mapper	`CacheMapper.java` , `sample_web_logs.txt`
Member 2 — Task 3 Reducer & Driver Lead	CacheReducer + CacheDriver developer	Write `CacheReducer.java` with multi-field aggregation; write `CacheDriver.java` ; configure DistributedCache in driver; verify combiner decision (not applicable — document why); write `compile_and_run.sh`	`CacheReducer.java` , `CacheDriver.java` , `scripts/` for task 3
Member 3 — Task 3 Testing & Validation	QA and data prep	Prepare `url_categories.txt` with realistic URL patterns; generate/download the web log dataset; upload to HDFS; run full-scale test; validate output; write Task 3 README and `expected_output.txt`	`url_categories.txt` , `setup_hdfs.sh` , `task3/README.md`
Member 4 — Task 11 Mapper & Reducer Lead	EmployeeMapper + DepartmentReducer developer	Write `EmployeeMapper.java` ; implement salary validation and error handling; write `DepartmentReducer.java` with total/avg/count logic; consider and implement Combiner	`EmployeeMapper.java` , `DepartmentReducer.java`
Member 5 — Task 11 Partitioner & Driver Lead	DepartmentPartitioner + Driver developer	Write `DepartmentPartitioner.java` (adapting `partitioner.txt`); implement `Configurable` interface; write `DepartmentDriver.java` ; set correct `numReduceTasks` ; inject department list into Configuration	`DepartmentPartitioner.java` , `DepartmentDriver.java`

Partitioner & Driver Lead	developer	write `DepartmentDriver.java`, set correct `numReduceTasks`, inject department list into Configuration	`DepartmentDriver.java`
Member 6 — Task 11 Testing, Docs & Integration	QA, data prep, GitHub manager	Prepare/download employee dataset (≥1GB); upload to HDFS; run full-scale test; validate multi-reducer output; write Task 11 README; manage GitHub repo (branching, PRs, merge conflicts); write overall `docs/`	`task11/README.md`, all `docs/`, `shared/`, `.gitignore`

Workflow Between Members

```
Week 1:
  Members 1,2,4,5 → Write code independently on their branches
  Members 3,6    → Prepare datasets, set up HDFS, write shell scripts

Week 2:
  Members 1+2    → Integrate Task 3 components, test on sample data
  Members 4+5    → Integrate Task 11 components, test on sample data
  Members 3+6    → Run full-scale tests, document results

Week 3:
  All → Code review, fix bugs, finalize documentation
  Member 6 → Final GitHub merge to main, submission packaging
```

---

8. DATASETS (1 GB+)

8.1 Task 3 — Web Log Datasets

You need web server access logs with fields matching: `requestId, urlPath, responseTimeMs, statusCode`

Option A: NASA HTTP Access Logs (BEST CHOICE for Task 3)

Source: NASA Kennedy Space Center web server logs from 1995, publicly archived.

Download URLs:

- `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz` (~21 MB compressed, ~200 MB uncompressed)
- `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Aug95.gz` (~25 MB compressed, ~300 MB uncompressed)
- Also available at: `https://www.kaggle.com/datasets/shawon10/web-log-dataset`

Format:

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

Why it fits Task 3: Contains URL paths and HTTP status codes. You'll need to adapt your Mapper to parse the standard Apache Combined Log Format and extract the URL path and status code. You can preprocess it to add `requestId` and `responseTimeMs`, or adapt the Mapper to parse the standard format.

To reach 1 GB: Combine July + August logs and replicate the file 2-3 times with `cat nasa_jul.txt nasa_aug.txt nasa_jul.txt > web_logs_1gb.txt`

Option B: Wikimedia Access Logs

Source: Wikimedia Foundation publishes hourly web traffic logs.

Download URL: `https://dumps.wikimedia.org/other/pageviews/`

Format: `en Wikipedia_Main_Page 12345 0`

Each file is 50-200 MB. Downloading ~10 hours of logs gives you 1+ GB easily.

Why it fits: Large volume, real URL-like page names, easily categorizable (by language, namespace, etc.). You'd treat the page name as the URL path.

Option C: CAIDA Anonymized Internet Traces

Source: CAIDA (Center for Applied Internet Data Analysis) — requires free registration.

URL: `https://www.caida.org/catalog/datasets/passive_dataset/`

Size: Several GB per trace.

Why it fits: Real network traffic data, very large scale.

Option D: Generate Synthetic Web Logs (MOST FLEXIBLE)

Generate your own 1GB log file with a Python script:

```python
import random, uuid, time

urls = ["/api/products", "/api/orders", "/api/users",
        "/home", "/about", "/contact",
        "/static/logo.png", "/static/style.css",
        "/admin/dashboard", "/admin/users",
        "/checkout", "/cart"]

statuses = [200]*70 + [301]*5 + [304]*10 + [400]*5 + [404]*7 + [500]*3

with open("web_logs_1gb.txt", "w") as f:
    count = 0
    while True:
        req_id = f"R{count:010d}"
        url = random.choice(urls)
        response_time = random.randint(10, 3000)
        status = random.choice(statuses)
        f.write(f"{req_id},{url},{response_time},{status}\n")
        count += 1
        if count % 1_000_000 == 0:
            size_gb = f.tell() / (1024**3)
            if size_gb >= 1.0:
                break

print(f"Generated {count:,} lines")
```

This gives you perfect control over categories and is immediately compatible with the exact format the task expects.

8.2 Task 11 — Employee Datasets

You need employee records with: `employee_id, department, salary, position, hire_date`

Option A: US Federal Employee Salary Data (BEST CHOICE for Task 11)

Source: US Office of Personnel Management (OPM) — completely public.

Download URL: `https://www.opm.gov/data/datasets/`
Direct dataset page: `https://fedscope.opm.gov/`

What you get: Name, agency, salary, grade, position, work schedule — millions of records, real-world, multiple departments/agencies.

Size: Multiple CSV files, combined easily exceeds 1 GB.

Format adaptation: Map "Agency" → "Department", "Pay Rate" → "Salary"

repo1 / Claude.ipynb

Top

Preview | Code | Blame | 981 lines (981 loc) · 46.7 KB

Raw

Download URL: `https://www.dol.gov/agencies/eta/foreign-labor/performance`

What you get: Employer name, job title, wage offered, SOC code, state — hundreds of thousands of records per year.

Size: Each year file is ~100-300 MB. Download 4-5 years to exceed 1 GB.

Format example:

```
CASE_NUMBER,EMPLOYER_NAME,JOB_TITLE,PREVAILING_WAGE,SOC_NAME,WORKSITE_STATE
I-200-19001-123456,Google LLC,Software Engineer,150000,Computer Occupations,CA
```

Map `SOC_NAME` (occupational group) → `department`, `PREVAILING_WAGE` → `salary`

Direct link (FY2023): `https://www.dol.gov/sites/dolgov/files/ETA/oflc/pdfs/LCA_Disclosure_Data_FY2023_Q4.xlsx`

Option C: Kaggle HR Analytics Datasets

URL: `https://www.kaggle.com/datasets/rhuebner/human-resources-data-set` **Also:** `https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset`

These are clean, well-formatted employee datasets. The IBM one is smaller (~1500 rows), but the HRDataset on Kaggle has more rows. To reach 1GB, generate synthetically using these as a template.

Option D: Generate Synthetic Employee Data (MOST FLEXIBLE)

```python
import random, csv

departments = ["IT", "HR", "Sales", "Finance", "Marketing", "Engineering", "Legal", "Operations"]
positions = {
    "IT": ["Developer", "Senior Developer", "DevOps Engineer", "Data Engineer"],
    "HR": ["HR Manager", "HR Assistant", "Recruiter", "Payroll Specialist"],
    "Sales": ["Sales Executive", "Sales Manager", "Account Manager"],
    "Finance": ["Accountant", "Financial Analyst", "CFO", "Auditor"],
    "Marketing": ["Marketing Manager", "Content Writer", "SEO Specialist"],
    "Engineering": ["Mechanical Engineer", "Civil Engineer", "Project Manager"],
    "Legal": ["Lawyer", "Legal Assistant", "Compliance Officer"],
    "Operations": ["Operations Manager", "Logistics Coordinator", "Analyst"]
}

with open("employees_1gb.csv", "w", newline='') as f:
```

```
    writer = csv.writer(f)
    count = 0
    while True:
        dept = random.choice(departments)
        emp_id = f"E{count:010d}"
        salary = random.randint(30000, 150000)
        position = random.choice(positions[dept])
        year = random.randint(2010, 2023)
        month = random.randint(1, 12)
        day = random.randint(1, 28)
        hire_date = f"{year}-{month:02d}-{day:02d}"
        writer.writerow([emp_id, dept, salary, position, hire_date])
        count += 1
        if count % 500_000 == 0:
            size_gb = f.tell() / (1024**3)
            if size_gb >= 1.0:
                break

print(f"Generated {count:,} employee records")
```

8.3 Dataset Compatible with Both Tasks?

Web logs and employee records are structurally very different, so a single real-world dataset that naturally covers both tasks is unlikely. However:

- **The NASA logs** can be used as the basis of Task 3 and a **synthetic employee dataset** for Task 11, keeping things manageable.
- **Alternatively**, if you use the **H-1B dataset**, the `SOC_NAME` field can act as a "department" for Task 11, AND if you treat the `EMPLOYER_NAME` URLs or case numbers as "URL paths" with response-time fields added synthetically, you can use it loosely for Task 3.

Recommendation: Use two different datasets — one per task — as they serve different analytical purposes.

QUICK REFERENCE SUMMARY

	Task 3	Task 11
Pattern	Hadoop DistributedCache + CacheMapper	Custom Partitioner
Key class to write	CacheMapper, CacheReducer, CacheDriver	DepartmentPartitioner, EmployeeMapper, DepartmentReducer, DepartmentDriver
Inspired by your materials	HadoopDistributedCache slides	`partitioner.txt` (LogProcessorMonthPartitioner)
Custom Writable?	No (Text keys and values are sufficient)	Optional (can adapt StringPairWritable)
Combiner?	No (avg is not associative)	Yes (sum is associative)
Number of Reducers	1 (as required)	N = number of departments
Output	1 file: category stats	N files: one per department
Best dataset	NASA logs or synthetic web logs	OPM federal data or synthetic employees
Dataset size	≥ 1 GB	≥ 1 GB